AI WEATHERWISE

Team No: 2

Team Head: KANIHA P

(0AF38D369E9013AAF5C4EFB47992279C)

Team Members: KANISHKA L

(B745F8DFC72D1ABB8C9443D476ACBA72)

Team Members: KARKULAZHI E U

(10E1517889E13B696084833B07D26249)

Team Members: KEERTHIKA B

(9B8AD90F0D903D9BA4A4B6AF19CC0F64)

Introduction

AI WeatherWise is a weather information platform designed to deliver real-time weather metrics together with AI-generated summaries and personalized recommendations. The backend is built with Node.js and Express.js, while MongoDB and Mongoose are used for data persistence. Google Gemini AI supports natural-language weather insights.

People may need weather information while travelling, planning outdoor activities, or choosing suitable clothing. Raw weather measurements can be difficult to interpret quickly. AI WeatherWise brings weather data and understandable guidance together in one service.

The platform also provides secure account access and favorite-location management. Registered users can save cities, retrieve their current weather, and request AI insights. Public weather endpoints can provide current metrics without requiring an account.

Objectives of the Project

The main objective of AI WeatherWise is to develop a centralized backend service that retrieves weather information and turns current environmental conditions into useful, easy-to-understand insights.

Specific Objectives

- Retrieve current weather metrics for a requested city or location.
- Allow registered users to create, view, update, and delete favorite locations.
- Generate natural-language weather summaries using Google Gemini AI.
- Provide personalized activity and weather-related recommendations based on live conditions.
- Protect user accounts and private operations through JWT authentication and password hashing.
- Maintain structured user and location data using MongoDB and Mongoose.
- Support API verification and service monitoring through developer tools.

The project also demonstrates modular backend development, database modeling, API routing, authentication, and integration with an external AI service.

System Overview

AI WeatherWise is organized as a RESTful backend application. API clients send HTTP requests to the Express server, which routes each request to the appropriate controller. Controllers apply application logic, communicate with database models, and return JSON responses.

Main System Components

- **API Interface:** Postman or similar API tools send requests to the defined endpoints and inspect JSON responses.
- **Express Server Gateway:** Handles network requests, maps GET, POST, PUT, and DELETE methods, parses JSON payloads, and configures CORS.
- **Authentication Middleware:** Reads JWT tokens, validates the user context, and protects routes that require authorization.
- **Controller and Service Logic:** Coordinates user operations, favorite-location management, weather retrieval, and AI insight requests.
- **Database Interface:** Mongoose connects application models and schemas with MongoDB collections.
- **AI Service Integration:** Google Gemini generates natural-language summaries and recommendations from weather conditions.

The architecture separates routing, controllers, middleware, models, and utility modules to support maintainability and scalability.

System Requirements

Hardware Requirements

- Processor: Intel Core i5 (8th Generation or above), AMD Ryzen 5, or equivalent.
- RAM: 8 GB minimum; 16 GB recommended for concurrent database and application processes.
- Storage: At least 1 GB of available disk space.

Software Requirements

- Operating System: Windows 10/11, macOS, or Linux.
- Node.js version 16 or above.
- npm version 8 or above.
- MongoDB database and Mongoose ODM.
- Google Gemini AI SDK for natural-language insight integration.
- Visual Studio Code or a similar code editor.
- Postman for API testing and verification.

These requirements provide the runtime, development, database, and testing environment needed to configure and execute the backend application.

Project Architecture

The application follows the Model-View-Controller (MVC) pattern. Its layers separate data structure, request processing, and API routing. In this headless backend, the interface is provided through API routes rather than traditional page templates.

Model Layer

The model layer defines Mongoose schemas for User and Location records. Schemas specify fields, data types, required values, defaults, and references before data is stored in MongoDB.

Controller Layer

Controllers receive requests from routes, validate request requirements, coordinate database and service operations, and prepare JSON responses for the client.

View / API Routing Layer

The API routing layer maps HTTP request paths and methods to controller handlers. It serves as the communication interface between API clients and backend logic.

Database and AI Services

Mongoose translates application operations into MongoDB queries. The Gemini integration provides weather summaries and recommendations based on current weather conditions.

Database Design and ER Relationship

The data model contains User and Location entities. A registered user may save multiple favorite locations, so the relationship between User and Location is one-to-many.

User Entity

- **_id:** ObjectId primary key.
- **name:** Required string.
- **email:** Required, unique string.
- **password:** Required hashed password string.
- **role:** String with default value “reader”.

Location Entity

- **_id:** ObjectId primary key.
- **user:** Reference to the User collection.
- **city:** Required string.
- **country:** Required string.
- **createdAt:** Date with current-time default.

Relationship

One User can be associated with multiple Location records. Each saved Location references its owning User, allowing favorite places to be managed in the context of the account.

System Workflow

The typical workflow for a registered user is as follows:

- The user registers or logs in.
- The user saves a favorite city through the API.
- For protected operations, the backend validates the JWT token.
- The user requests current weather metrics for the selected city.
- The user requests an AI summary or recommendation based on the weather data.
- The backend processes the request and returns the requested information in JSON format.

The system uses the saved location records to organize user preferences and the weather service integration to obtain conditions for insight generation.

Backend Development

The backend is developed with Node.js, Express.js, and MongoDB using a modular structure. Routing, controllers, middleware, models, and utility modules are kept separate to make the application easier to maintain.

Application Entry Point: index.js

The index.js file is the main entry point. It loads environment variables, initializes the Express server, configures middleware, establishes the MongoDB connection, registers API routes, and starts listening for requests after the database connection succeeds.

Request Processing

Incoming requests are mapped to route handlers. Middleware performs shared tasks such as parsing JSON, configuring CORS, and validating authentication tokens where required. Controllers coordinate the requested operation and return JSON output.

Conclusion

AI WeatherWise combines weather data retrieval, favorite-location management, secure authentication, and AI-generated insights in a modular backend service. Node.js and Express.js provide the server and routing framework, MongoDB and Mongoose manage structured data, and Google Gemini supports natural-language weather summaries and recommendations.

The project organizes its functions through MVC-inspired layers and REST APIs. It provides users with a way to save locations, retrieve weather metrics, and request context-aware suggestions, while also including authentication, request sanitization, centralized error handling, and fallback behavior described in the source material.